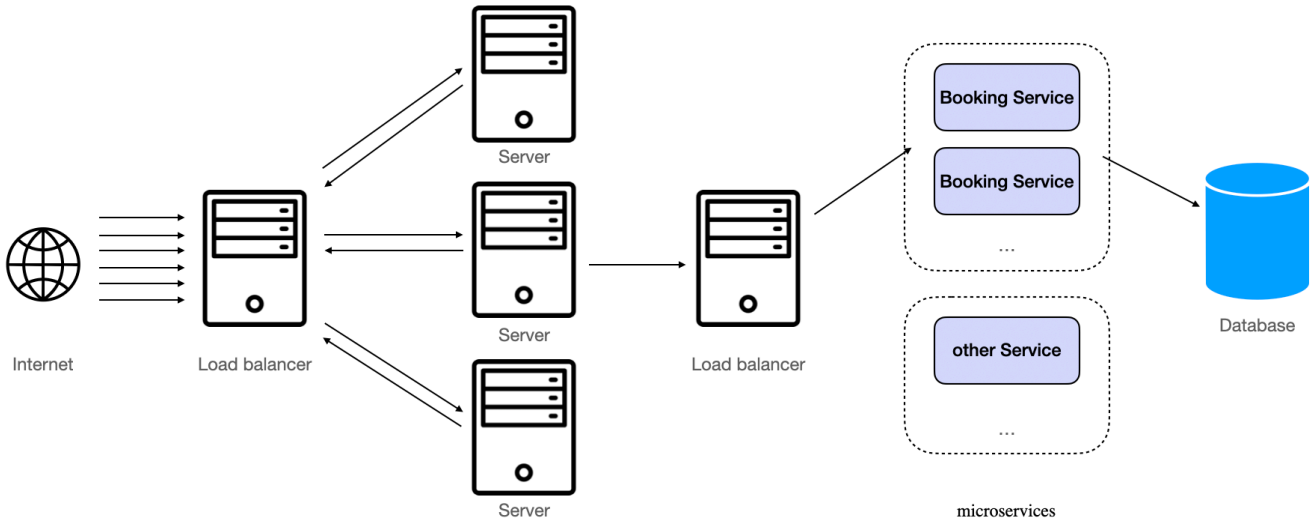
From One-to-One to Pub/Sub and Beyond

systemdesignschool.io/fundamentals/message-queue-use-cases



Message Queue Use Cases and Patterns

We already covered the basics of message queues in the last article. Let's look at some patterns for message queues:

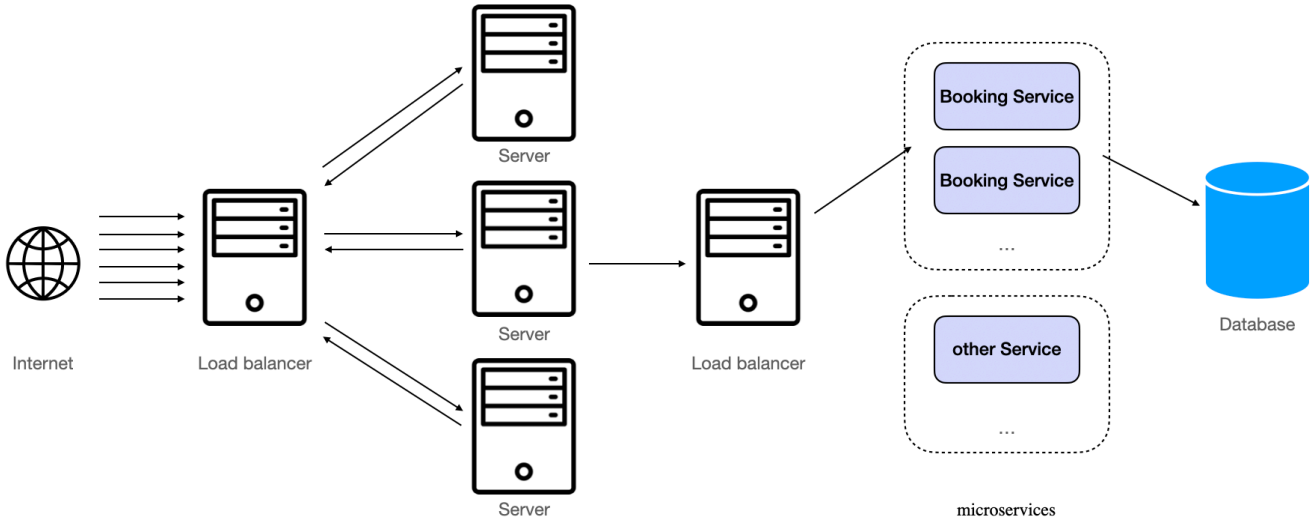
1. One-to-one message queue: In this pattern, each message is processed by one consumer.
    - Producers typically know the destination they are sending messages to.
    - Example: AWS Simple Queue Service (SQS) is an example of a one-to-one message queue. In SQS, each message is sent to a specific queue and is processed by only one consumer.
2. Publish/Subscribe (pub/sub) messaging: In this pattern, each message can be processed by more than one consumer, following the "fanout" design pattern.
    - Producers send messages to a topic.
    - There is a central topic to which messages are published.
    - Each subscriber receives a copy of the message.
    - Messages are sent immediately to the consumers.
    - Producers typically don't know the destinations they are sending messages to, as they only publish messages to the topic.
    - Example: AWS Simple Notification Service (SNS) and SQS can be combined to implement a pub/sub messaging system. In this setup, SNS acts as the topic to which messages are published, and SQS queues act as subscribers. When a message is published to an SNS topic, a copy is sent to each subscribed SQS queue, allowing multiple consumers to process the message.

We recommend you try out the Redis-queue exercise in the next section to get a feel of how message queues work before continuing with the following content.

Let’s look at some concrete examples.

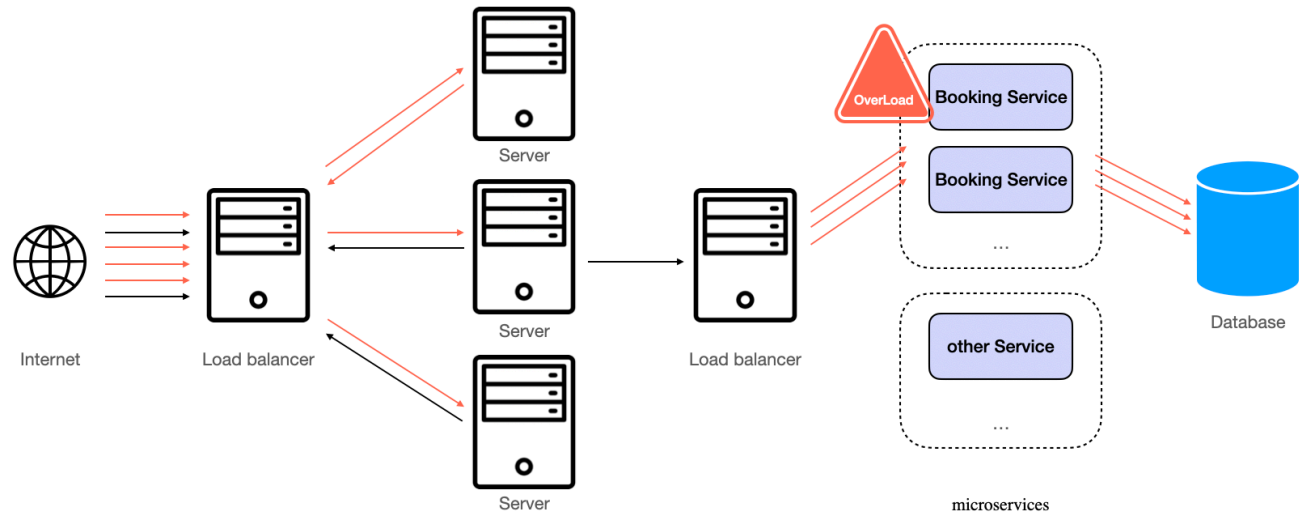
Server to Service Data Flow Through Message Queues

Imagine a hotel booking service where customers connect through a website. The traffic from the customers is routed through a load balancer, which directs the requests to a group of web servers handling the traffic. When someone makes a hotel booking, the request is sent to a second layer of microservices that deal with bookings. This second layer sits behind another load balancer, and the booking service itself uses a relational database as its datastore.



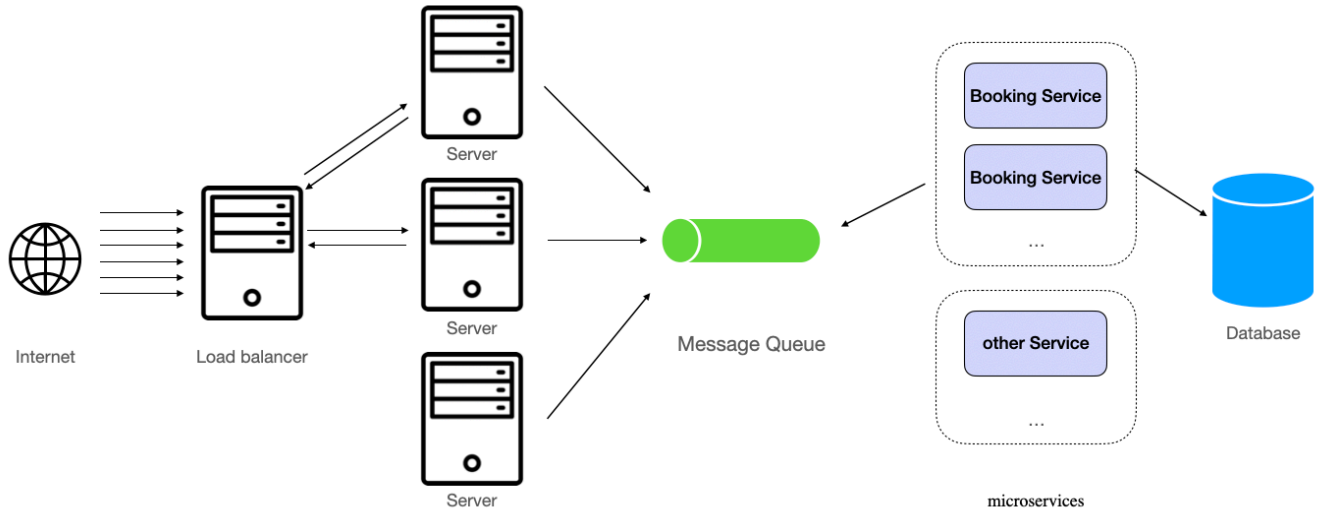
Server to Service Data Flow Without Message Queues

If you examine the system closely, most of the website traffic is from users browsing offers, with fewer requests coming from actual bookings. Now, let's say you launch a large promotional campaign, and your customers start booking hotels at a rate your system is unprepared to handle. The numerous web servers could send a high volume of requests to the second layer of booking services, which might not be equipped to handle the increased load. In this scenario, your relational database could start dropping or timing out on requests, leading to errors being propagated back to your customers.



Server Overload Without Message Queues

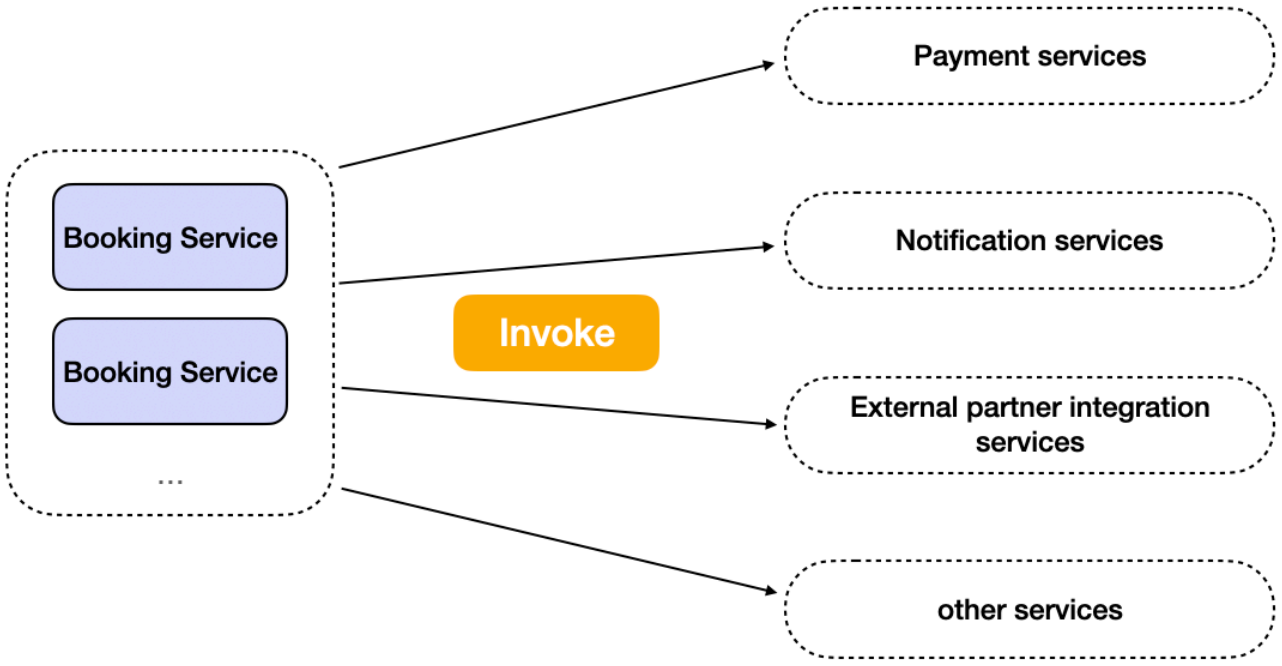
One solution to this problem, where messaging comes in handy, is to replace the load balancer with a queue. This queue can absorb incoming traffic, and the booking services behind it, which communicate with the database, can consume the messages in the backlog at a rate they can handle comfortably. This approach helps protect your system from capacity mismatches between layers, preventing one layer from spiking in traffic while the other struggles to keep up.



Server to Service Data Flow Through Message Queues

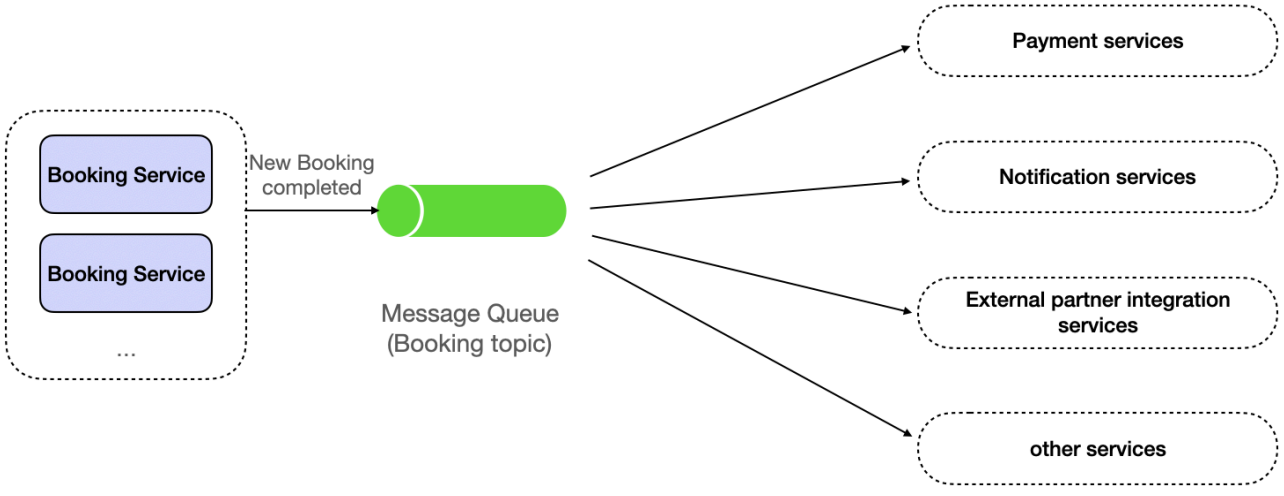
Use Message Queue and Pub/Sub to Notify Consumers

A microservice can talk to another service directly. For example, after a booking is complete, a booking service can invoke the notification service to send a mobile push notification to the customer and ask the email service to send a voucher to the customer, etc.



Invoking other microservices by booking services

Different microservices may also experience overload due to response speed, request frequency, and sudden increases in traffic, thereby reducing the overall performance of the system and even causing it to crash. In this case, message queues can also be used to decouple between microservices.



Using message queue as a bridge between microservices

Asynchronous Background Tasks Using Message Queues

Now let's say the booking service has to complete a number of tasks after a successful booking. Here's a table showing the time it'd take to do these tasks

Task	Time (ms)
Update booking status in the database	30
Create a PDF voucher	1000
Send a confirmation email with the voucher attached	500
Notify other services about the success	1000

If we do these tasks synchronously, we'd have 30 + 1000 + 500 + 1000 = 2530ms which is quite slow.

```
def update_booking_status(): ... def create_pdf_voucher(): ... def
send_confirmation_email(): ... def notify_other_services(): ...
def synchronous_tasks(): update_booking_status() create_pdf_voucher()
send_confirmation_email() notify_other_services()
```

We could only update booking status in the database synchronously and do the rest of the work asynchronously in background threads. This way the response time is only 30ms.

```
from threading import Thread # Asynchronous using a background thread def
async_thread_task(task): task() def asynchronous_tasks():
update_booking_status() tasks = [ create_pdf_voucher, send_confirmation_email,
notify_other_services, ] threads = [Thread(target=async_thread_task, args=
(task,)) for task in tasks] [thread.start() for thread in threads]
[thread.join() for thread in threads]
```

Sync Task	Time (ms)
Update booking status in the database	30
Async Tasks	
Create a PDF voucher	1000
Send a confirmation email with the voucher attached	500
Notify other services about the success	1000

This works, however, background threads could fail and we can't really know if the tasks succeeded or not.

The better approach is to put the asynchronous tasks into a task queue and have workers finish the tasks. When a worker finishes a task, it deletes it from the queue. We can poll the queue for the task to know whether it's done or not. Here's an example implementation using Redis-queue.

```
import os import time from rq import Queue from redis import Redis from rq.decorators import job redis_conn = Redis() # Tasks @job('default', connection=redis_conn) def update_booking_status(): time.sleep(0.03) print("Booking status updated.") @job('default', connection=redis_conn) def create_pdf_voucher(): time.sleep(1) print("PDF voucher created.") @job('default', connection=redis_conn) def send_confirmation_email(): time.sleep(0.5) print("Confirmation email sent.") @job('default', connection=redis_conn) def notify_other_services(): time.sleep(1) print("Other services notified.") if __name__ == "__main__": # Synchronously update booking status first update_booking_status() # Asynchronously execute other tasks tasks = [ create_pdf_voucher, send_confirmation_email, notify_other_services, ] # Enqueue tasks [task.delay() for task in tasks] # Note: You'll need to run a separate RQ worker process to execute the tasks. # Run the following command in your terminal: rq worker
```

Dead letter queue

And finally, we should mention dead letter queues. A dead letter queue (DLQ) is a special type of queue that is used to store messages that could not be processed or delivered by the main message queue. These messages are often referred to as "dead letters" because they cannot be processed for some reason, such as a failure in the consumer application, an invalid message format, or exceeding the maximum number of delivery attempts.

Dead letter queues are useful for several reasons:

1. Error handling: When a message cannot be processed or delivered, it is moved to the dead letter queue instead of being lost or repeatedly retried, which could cause other issues.
2. Monitoring and debugging: By examining the messages in the dead letter queue, developers can identify issues in the message processing logic or the message itself, helping them debug and fix any problems in the system.
3. Isolation: The dead letter queue isolates problematic messages from the main message queue, preventing them from affecting the processing of other valid messages.
4. Retry and reprocessing: In some cases, messages in the dead letter queue can be reprocessed and returned to the main queue once the issue has been resolved.

Consumer failures

If a consumer fails to process a message, the appropriate action typically depends on the nature of the failure and the specific requirements of your application.

Here are several strategies that you might consider:

1. **Retries with Backoff and Jitter**: Retry the operation after a delay. This is useful if the failure might be transient, such as a temporary network issue. To avoid overwhelming your system with retries or retrying at the same time as other consumers (known as the "thundering herd problem"), you can use a backoff strategy to increase the delay between retries, and jitter to randomize the delay.

2. **Dead Letter Queues**: Send the message to a separate "dead letter" queue instead of re-enqueuing it in the main queue. This keeps the main queue flowing and allows you to inspect and handle failed messages separately. Dead letter queues are often combined with a limit on the number of processing attempts.
3. **Message Discarding**: In some cases, you might choose to simply discard failed messages, especially if they are not critical.